

Universidad de San Carlos de Guatemala

Centro Universitario de Occidente

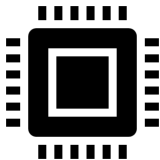
División de Ciencias de la ingeniería

Ingeniería

Estructura de Datos Sección “A”

Ing. Mario Moises Ramirez Tobar

Primer Semestre 2026



Estudiante:

Pablo Alejandro Maldonado de León

Carné: 202430233

TAD'S Proyecto #2



Descripción:

En el siguiente documento se le presentaron todos los TAD 'S utilizados para poder realizar el primer proyecto del curso de estructura de datos. En este caso se utilizaron muchos signals debido a la complejidad que tenía de intercomunicación entre pantallas y acciones requeridas por el sistema.

TAD'S PRINCIPALES DEL BACKEND

Estos son la parte principal del funcionamiento de la aplicación ya que definen por completo cuáles son los métodos principales de cada estructura de datos.

También es importante destacar que es bastante corto respecto a todo lo esperado y esto ocurre porque simplemente se delegan tareas a cada clase y cada clase se especializa en su respectiva tarea. No está demás decir que también hay una clase con muchos métodos ya que esta es utilizada como un controlador y por cuestiones de la complejidad que implica delegar a más controladores su respectivo rol se optó por agrupar todo en un solo controlador. Siendo de esta forma la clase más grande del proyecto. Sin embargo esto es por el propio aprendizaje que se genera que a veces ya es funcional cuando se logran ver alternativas. Sin embargo se buscó la mayor cohesión posible bajo estas limitaciones que tenía la herramienta Qt Creator.

LINK AL DIAGRAMA DE CLASES EN EL QUE SE BASÓ EL BACKEND DEL PROYECTO:

<https://app.diagrams.net/#G1BCIDfNIgt-QoM5RCRk9x7FZVSzvTnGJo#%7B%22pageId%22%3A%22ZyCEvR6T124IhAAXozIO%22%7D>

ArbolAvl

Interfaz Pública:
ArbolAvl()
~ArbolAvl()
void insertar(Producto producto)
ListaEnlazada<Producto> obtenerProductosOrdenados()
ListaEnlazada<Producto> buscarProducto(const std::string& nombre)
void vaciarArbol(NodoAvl* nodo)
void eliminarPorCodigo(const std::string &codigo)
NodoAvl* getRaiz()
std::string generarDot()

Implementación Privada:
NodoAvl* raiz
int obtenerAltura(NodoAvl* nodo)
int obtenerFactorBalance(NodoAvl* nodo)
NodoAvl* rotacionDerecha(NodoAvl* y)
NodoAvl* rotacionIzquierda(NodoAvl* x)
NodoAvl* insertarRecursivo(NodoAvl* nodo, Producto producto)
bool existeCodigoBarra(NodoAvl* nodo, const std::string& codigo)
Producto * buscarPorCodigobarra(NodoAvl* nodo, const std::string &codigo)
NodoAvl* nodoMinimo(NodoAvl* nodo)
NodoAvl* eliminarRecursivo(NodoAvl* nodo, const std::string &nombre, const std::string& codigo)
void generarDotRecursivo(NodoAvl* nodo, std::stringstream& sStream)
void buscarProductoRecursivo(NodoAvl* nodo, const std::string& nombre, ListaEnlazada<Producto>& lista)
void listarProductosInOrden(NodoAvl* nodo, ListaEnlazada<Producto>& lista)

Debido a que la clase **ArbolAvl** utiliza por asociación un nodo especial para esta estructura se adjunta la clase justo debajo del TAD del árbol AVL.

NodoAvl

Interfaz Pública:
NodoAvl(const Producto &_producto);
NodoAvl * getIzquierda() const
NodoAvl * getDerecha() const
void setIzquierda(NodoAvl* _izquierda)
void setDerecha(NodoAvl* _derecha)
int getAltura() const
void setAltura(int _altura)
Producto& getDato()
const Producto& getDato() const

Implementación Privada:
Producto producto
NodoAvl* izquierdo
NodoAvl* derecho
int altura

ArbolB

Esta es la segunda estructura de datos más grande utilizada. Y sigue el mismo principio de poder almacenar datos de forma más eficiente que una lista enlazada.

Interfaz Pública:
ArbolB(int _t)
~ArbolB()
void insertar(const Producto &producto)
ListaEnlazada<Producto> buscarRango(const std::string& inicio, const std::string& fin)
void eliminar(const std::string& codigo)
std::string generarDot()
NodoB * getRaiz()

Implementación Privada:
NodoB * raiz;
int t;
void dividirHijo(NodoB* padre, int i, NodoB* lleno);
void insertarNoLleno(NodoB* nodo, const Producto &producto);
void insertarNuevo(const Producto &producto);
bool existeCodigo(NodoB* nodo, const std::string &codigo);
void buscarRangoAux(NodoB* nodo, const std::string& inicio, const std::string& fin, ListaEnlazada<Producto>& resultados)
void eliminarAux(NodoB* nodo, const std::string& fecha, const std::string& codigo)
int buscarClave(NodoB* nodo, const std::string& fecha, const std::string& codigo)
void llenar(NodoB* nodo, int indice)
void prestarDeAnterior(NodoB* nodo, int indice)
void prestarDeSiguiente(NodoB* nodo, int indice)
void fusionar(NodoB* nodo, int indice)

void eliminarDeNodoInterno(NodoB* nodo, int idx)
Producto getPredecesor(NodoB* nodo, int idx)
Producto getSucesor(NodoB* nodo, int idx)
void generarDotRecursivo(NodoB* nodo, std::stringstream& sStream)
Producto buscarPorCodigoExhaustivo(NodoB* nodo, const std::string& codigo, bool& encontrado)

Debido a que la clase **ArbolB** utiliza por asociación un nodo especial para esta estructura se adjunta la clase justo debajo del TAD del árbol B.

NodoB

Interfaz Pública:
NodoB(bool _hoja)
~NodoB()
ListaEnlazada<Producto>& getClaves()
ListaEnlazada<NodoB*>& getHijos()
bool getEsHoja()

Implementación Privada:
bool esHoja
ListaEnlazada<Producto> claves
ListaEnlazada<NodoB*> hijos

ArbolBMas

Esta es la tercera estructura de datos más grande utilizada. Y sigue el mismo principio de poder almacenar datos de forma más eficiente que una lista enlazada e incluso superando en tiempos de búsqueda a las demás estructuras.

Interfaz Pública:
ArbolBMas(int _orden)
~ArbolBMas()
void insertar(const Producto &producto)
void eliminar(const std::string& codigoBarra)
NodoBMas* getRaiz()
ListaEnlazada<Producto> buscarPorCategoria(const std::string &categoriaBuscada)
std::string generarDot()

Implementación Privada:
NodoBMas* raiz
int orden
void insertarEnNodo(NodoBMas* nodo, const Producto &producto)
void dividirHoja(NodoBMas* hoja, NodoBMas* padre, int indiceEnPadre)
void dividirInterno(NodoBMas* interno, NodoBMas* padre, int indiceEnPadre)
void insertarRecursivo(NodoBMas *nodo, NodoBMas *padre, Producto producto, int indiceEnPadre)
bool yaExiste(NodoBMas* nodoRaiz, const Producto &producto)
void generarDotRecursivo(NodoBMas* nodo, std::stringstream& sStream)
bool eliminarRecursivo(NodoBMas* nodo, NodoBMas* padre, int indiceEnPadre, const Producto& producto)
void balancearNodo(NodoBMas* padre, NodoBMas* hijo, int indiceHijo)
void fusionarNodos(NodoBMas* padre, int indiceSeparador, NodoBMas* izquierdo, NodoBMas* derecho)

Debido a que la clase **ArbolBMas** utilizado por asociación un nodo especial para esta estructura se adjunta la clase justo debajo del TAD del árbol B+.

NodoBMas

Interfaz Pública:
NodoBMas(bool _esHoja)
~NodoBMas()
ListaEnlazada<Producto>& getClaves()
ListaEnlazada<NodoBMas*>& getHijos()
bool getEsHoja()
NodoBMas * getSiguiente()
void setSiguiente(NodoBMas* _siguiente)

Implementación Privada:
bool esHoja
ListaEnlazada<Producto> claves
ListaEnlazada<NodoBMas*> hijos
NodoBMas* siguiente

ListaEnlazada

Esta fue una estructura genérica que permite manejar los diferentes tipos de clases o punteros utilizados en el propio proyecto. No está demás agregar que esta lista fue lo más fiel a una lista doblemente enlazada solo que utilizada en puntos específicos de forma lineal y otras si haciendo las búsquedas con su debida complejidad ya que todas las estructuras se comparaban a la lista enlazada.

Interfaz Pública:
ListaEnlazada()
~ListaEnlazada()
void insertarAtras(const T&valor)
void insertarFrente (const T&valor)
void removerAtras()
void removerFrente()
bool esVacia() const
int getLongitud()const
void limpiar()
void insertar(int pos, const T& valor)
void eliminar(int pos)
T& getValor(int pos)
void setValor(int posicion, const T& nuevoValor)
Nodo<T>* getCabeza() const
T obtenerFrente() const

Implementación Privada:
Nodo<T> * cabeza
Nodo<T> * cola
int longitud
Nodo<T>* getNode(int pos) const

NodoLista

Esta es la clase que utiliza esta lista para almacenar datos, es genérica para poder manejar la lógica de las listas enlazadas de todas las estructuras que utilizan a la lista enlazada por asociacion, sobre todo ya que estas listas poseían un doble enlace.

Interfaz Pública:
NodoLista(const T &producto)
NodoLista()
~NodoLista()
T& getDato()
Nodo<T>* getSiguiente() const
Nodo<T>* getAnterior() const
void setSiguiente(Nodo<T>* _siguiente)
void setAnterior(Nodo<T>* _anterior)
void setValor(const T& nuevoValor)

Implementación Privada:
T dato
Nodo<T> * siguiente
Nodo<T> * anterior

TablaHash

Interfaz Pública:
TablaHash(int capacidadInicial)
~TablaHash()
void insertar(const Producto& nuevoProducto)
Producto* buscar(const std::string& codigoBarra)
bool eliminar(const std::string& codigoBarra)
std::string generarDot()
int getCapacidad()
ListaEnlazada<Producto>** getTabla()

Implementación Privada:
ListaEnlazada<Producto>** tabla
int capacidad
int elementos
unsigned int calcularIndice(const std::string& clave)
void redimensionarHash()
void inicializarTabla()
Producto* buscarProducto(const std::string& codigoBarra)

Cola

Interfaz Pública:
Cola()
~Cola()
void encolar(Producto* prod)
Producto* desencolar()
std::vector<Producto*> obtenerCopiaElementos() const

Implementación Privada:
NodoCola* frente
NodoCola* final
mutable std::mutex candado

NodoCola

Es la clase de la que se apoya la cola para poder funcionar.

Interfaz Pública:
NodoCola(Producto* prod)
Producto* getProducto()
NodoCola* getSiguiente() const
void setSiguiente(NodoCola* _siguiente)

Implementación Privada:
Producto* producto
NodoCola* siguiente

Grafo

Eta es la clase principal que permite manejar toda la red principal de sucursales disponibles en la aplicacion.

Interfaz Pública:
Grafo()
~Grafo()
double obtenerTiempoArista(const std::string& idOrigen, const std::string& idDestino, bool porTiempo) const
void setSiguiente(NodoCola* _siguiente)
void insertarSucursal(Sucursal *nueva)
void modificarSucursal(const std::string &_id,const std::string &_nombre, const std::string &_ubicacion, double _ingreso, double _despacho, double _preparacion)
void eliminarSucursal(const std::string &id)
Sucursal* buscarSucursal(const std::string& id) const
void agregarConexion(const std::string &origen, const std::string &destino, double tiempo, double costo)
const std::vector<Sucursal*>& getNodos() const
const std::vector<std::vector<Conexion*>>& getMatriz() const
std::string generarGraphviz() const
std::vector<Sucursal*> obtenerRutaOptima(const std::string& origen, const std::string& destino, bool porTiempo) const

Implementación Privada:
std::vector<Sucursal*> nodos
std::vector<std::vector<Conexion*>> matriz
int obtenerIndice(const std::string& id) const

TAD DEL GESTOR PRINCIPAL DE LAS ESTRUCTURAS

Esta es la clase principal manejadora de todas las estructuras utilizadas para poder almacenar datos.

GestorEstructuras

Interfaz Pública:
GestorEstructuras()
~GestorEstructuras()
void insertarListasCsv(const std::string &nombre,const std::string &key,const std::string &categoria, const std::string &fecha, const std::string &marca, double precio, int stock)
void insertarArbolAvl(const std::string &nombre, const std::string &key, const std::string &categoria, const std::string &fecha, const std::string &marca, double precio, int stock)
void insertarArbolB(const std::string &nombre, const std::string &key, const std::string &categoria, const std::string &fecha, const std::string &marca, double precio, int stock)
void insertarArbolBMas(const std::string &nombre, const std::string &key, const std::string &categoria, const std::string &fecha, const std::string &marca, double precio, int stock)
void eliminarArbolAvl(const std::string &key)
void eliminarArbolB(const std::string &key)
void eliminarArbolBMas(const std::string &key)
void eliminarListaOrdenada(const std::string &key)
void eliminarListaNoOrdenada(const std::string &key)
void insertarListaNoOrdenada(const std::string &nombre,const std::string &key,const std::string &categoria, const std::string &fecha, const std::string &marca, double precio, int stock)
void insertarListaOrdenada(const std::string &nombre,const std::string &key,const std::string &categoria, const std::string &fecha, const std::string &marca, double precio, int stock)
void validarCsv(const QStringList& columnas, int fila)
void agregarErrorLista(const std::string mensaje, int fila)
QString generarContenidoLog()
ListaEnlazada<ErroresLectura>* getListaErrores()

bool tieneErrores() const
void generarListaOrdenada(int criterio)
void ordenarLista(int criterio)
void setCargoArchivo(bool carga)
bool getCargoArchivo()
ListaEnlazada<Producto> * getListaNoOrdenada()
std::string serializarListaCsv()
NodoAvl * getRaizArbolAvl()
NodoB * getRaizArbolB()
NodoBMas * getRaizArbolBMas()
std::string obtenerGraphvizAvl()
std::string obtenerGraphvizB()
std::string obtenerGraphvizBMas()
void validarInsercion(const std::string &_nombre,const std::string &_codigoBarra,const std::string &_categoria,const std::string &_fechaExpiracion,const std::string &_marca, const std::string &_precio, const std::string &_stock)
ListaEnlazada<Producto> buscarProductoAvl(const std::string& nombre)
ListaEnlazada<Producto> buscarProductoListaOrdenada(const std::string& nombre)
ListaEnlazada<Producto> buscarProductoListaNoOrdenada(const std::string& nombre)
ListaEnlazada<Producto> buscarProductoBMas(const std::string& categoria)
ListaEnlazada<Producto> buscarProductoCategoriaListaOrdenada(const std::string& categoria)
ListaEnlazada<Producto> buscarProductoCategoriaListaNoOrdenada(const std::string& categoria)
ListaEnlazada<Producto> buscarProductoB(const std::string &limiteInferior, const std::string &limiteSuperior)
ListaEnlazada<Producto> buscarProductoFechasListaOrdenada(const std::string &limiteInferior, const std::string &limiteSuperior)
ListaEnlazada<Producto> buscarProductoFechasListaNoOrdenada(const std::string &limiteInferior, const std::string &limiteSuperior)

ListaEnlazada<Producto> listarProductosAvl()
ListaEnlazada<Producto> listarProductosOrdenados()
ListaEnlazada<Producto> listarProductosNoOrdenados()
ListaEnlazada<Producto> getProductosExtremos()
ListaEnlazada<Producto> getProductosIntervalo(const Producto &productoInferior,const Producto &productoSuperior, int orden)
ListaEnlazada<Producto> getRangosAleatorios()
ListaEnlazada<Producto> getProductosExtremosIntervalo()

Implementación Privada:
ArbolAvl * arbolAvl = nullptr
ArbolB * arbolB = nullptr
ArbolBMas * arbolBMas = nullptr
ListaEnlazada<Producto> *listaNoOrdenada = nullptr
ListaEnlazada<Producto> * listaOrdenada = nullptr
ListaEnlazada<ErroresLectura> * listaErrores = nullptr
bool cargoArchivo
bool esFechaISO(const QString& fecha)
bool existeProductoListaNoOrdenada(const std::string &codigo)
bool existeProductoListaOrdenada(const std::string &codigo)

Excepciones

En este proyecto se hizo uso de la creacion de excepciones propias y gracias a estas clases se puede delegar la logica del modelo MVC ya que permiten reportar errores instantaneamente para activar los mecanismos que hay para poder cancelar una operacion.

DeleteException

Interfaz Pública:
explicit DeleteException(const std::string& msg)
virtual const char* what() const noexcept override

Implementación Privada:
std::string mensaje

InsertException

Interfaz Pública:
explicit InsertException(const std::string& msg)
virtual const char* what() const noexcept override

Implementación Privada:
std::string mensaje

ReaderCsvException

Interfaz Pública:
explicit ReaderCsvException(const std::string& msg)
virtual const char* what() const noexcept override

Implementación Privada:
std::string mensaje

Producto

Esta es la clase principal que representa a todos los productos que se pueden almacenar dentro de la aplicación para poder generar la interacción de toda la aplicación. No es una parte con muchos roles a cargos pero sí representa al objeto manejado en el sistema.

Interfaz Pública:
Producto()
Producto(const std::string &_nombre,const std::string &_codigoBarra, const std::string &_categoria, const std::string &_fechaExpiracion, const std::string &_marca, double _precio, int _stock)
std::string getNombre() const
std::string getCodigoBarra() const
std::string getCategoria()const
std::string getFechaExpiracion()const
std::string getMarca() const
double getPrecio() const
int getStock() const

Implementación Privada:
std::string nombre
std::string codigoBarra
std::string categoria
std::string fechaExpiracion
std::string marca
double precio
int stock

Sucursal

Esta es la clase principal que representa a todas las sucursales que se pueden almacenar dentro de la aplicación para poder generar la interacción de toda la aplicación. Y la navegación entre redes.

Interfaz Pública:
Sucursal(const std::string& id,const std::string& nombre,const std::string& ubicacion,double t_ingreso,double t_preparacion,double t_despacho)
~Sucursal()
std::string getId() const
std::string getNombre() const
std::string getUbicacion() const
double getTiempoIngreso() const
double getTiempoPreparacion() const
double getTiempoDespacho() const
GestorEstructuras* getAlmacen()
void setNombre(const std::string& n)
void setUbicacion(const std::string& u)
void setTiempoIngreso(double t)
void setTiempoPreparacion(double t)
void setTiempoDespacho(double t)

Implementación Privada:
std::string id
std::string nombre
std::string ubicacion
double tiempoIngreso
double tiempoPreparacion
double tiempoDespacho
GestorEstructuras* almacen

Conexion

Esta es la clase principal que representa a las conexiones entre sucursales y se toman como referencia para poder generar la matriz de adyacencia.

Interfaz Pública:
Conexion()
Conexion(double t,double c)
double getTiempo() const
double getCosto() const
bool getExiste() const
void setTiempo(double t)
void setCosto(double c)
void setExiste(bool e)

Implementación Privada:
double tiempo
double costo
bool existe

GestorEnvios

Esta es la clase más importante porque es la que permite manejar los hilos para poder crear las animaciones de que se van generando los viajes entre productos de las sucursales.

Interfaz Pública:
GestorEnvios(Grafo* _grafo)
~GestorEnvios()
void iniciarEnvio(Producto* paquete,const std::vector<Sucursal*>& ruta,bool porTiempo)
std::vector<HistorialEnvio> getHistorial()
void setRedGrafo(Grafo* _redGrafo)

Implementación Privada:
Grafo* redGrafo
std::vector<std::thread> hilosActivos
std::vector<HistorialEnvio> historial
std::mutex mutexHistorial
void procesarViaje(Producto* paquete,std::vector<Sucursal*> ruta,bool porTiempo)

ErroresLectura

Esta es la clase principal utilizada para poder reportar errores. Este objeto representa cualquier error que surja dentro de la aplicación. De esta forma se puede generar un log de errores que se puede imprimir.

Interfaz Pública:
ErroresLectura(const std::string &_mensaje,int _linea)
ErroresLectura()
std::string getMensaje()
int getLinea()

Implementación Privada:
std::string mensaje
int linea

Controlador

Esta es una de las clases principales del proyecto ya que en el proyecto se utilizó un pseudo modelo vista controlador en donde el backend es independiente al frontend por lo tanto se hizo este tipo de implementación para darle escalabilidad.

Interfaz Pública:
explicit Controlador(QObject *parent = nullptr)
~Controlador()
void evaluarEstadoCerrarCsv()
void verificarRefresco()
void actualizarDatosIngresados()
void ordenarListado(int orden)

Implementación Privada:
GestorEstructuras * gestorBackend = nullptr
void insertarListaCsv(const std::vector<QString> & data)
void insertarArbolAvlCsv(const std::vector<QString> & data)
void insertarArbolBCsv(const std::vector<QString> & data)
void insertarArbolBMasCsv(const std::vector<QString> & data)
void verificarErrores()
void insertarEnListas(const std::string &_nombre,const std::string &_codigoBarra, const std::string &_categoria, const std::string &_fechaExpiracion, const std::string &_marca, double _precio, int _stock)
void insertarEnArbolAvl(const std::string &_nombre,const std::string &_codigoBarra, const std::string &_categoria, const std::string &_fechaExpiracion, const std::string &_marca, double _precio, int _stock)
void insertarEnArbolB(const std::string &_nombre,const std::string &_codigoBarra, const std::string &_categoria, const std::string &_fechaExpiracion, const std::string &_marca, double _precio, int _stock)
void insertarEnArbolBMas(const std::string &_nombre,const std::string &_codigoBarra,

const std::string &_categoria, const std::string &_fechaExpiracion, const std::string &_marca, double _precio, int _stock)
void buscarAvlNombre(const std::string &nombre)
void buscarListasNombre(const std::string &nombre)
void buscarListasOrdenadaNombre(const std::string &nombre)
void buscarListasNoOrdenadaNombre(const std::string &nombre)
void buscarListasCategoria(const std::string &categoria)
void buscarListasOrdenadaCategoria(const std::string &categoria)
void buscarListasNoOrdenadaCategoria(const std::string &categoria)
void buscarBMasCategoria(const std::string &categoria)
void buscarBFecha(const std::string &limiteInferior, const std::string &limiteSuperior)
void buscarListasFecha(const std::string &limiteInferior, const std::string &limiteSuperior)
void buscarListaOrdenadaFecha(const std::string &limiteInferior, const std::string &limiteSuperior)
void buscarListaNoOrdenadaFecha(const std::string &limiteInferior, const std::string &limiteSuperior)
void eliminarEnListas(const std::string &_codigoBarra)
void eliminarEnArbolAvl(const std::string &_codigoBarra);
void eliminarEnArbolB(const std::string &_codigoBarra);
void eliminarEnArbolBMas(const std::string &_codigoBarra)
void listarAlfabeticoListas()
void listarAlfabeticoAvl()

Implementación de Signals:
void limpiarDatos()
void guardarArchivoCsv()
void procesarCsv(const std::vector<QString> & data)
void insercionProducto(const std::string &_nombre,const std::string &_codigoBarra, const std::string &_categoria, const std::string &_fechaExpiracion, const std::string &_marca, const std::string &_precio, const std::string &_stock)
void buscarPorNombre(const std::string &nombre)
void buscarPorCategoria(std::string categoria)
void buscarPorFecha(const std::string &limiteInferior, const std::string &limiteSuperior)
void eliminarProducto(const std::string &codigo)
void listarProductos()
void obtenerArbolAvl()
void obtenerArbolB()
void obtenerArbolBMas()
void generarGraphArbolAvl()
void generarGraphArbolB()
void generarGraphArbolBMas()
void prepararLogParaDescarga()
void pruebaAleatoriaFechas(int consultas,int veces, const std::string &limiteInferior, const std::string &limiteSuperior)
void pruebaExtremosFechas(int consultas,int veces)
void pruebaAleatoriaNombre(int consultas,int veces)
void pruebaExtremosNombre(int consultas,int veces)
void pruebaAleatoriaCategoria(int consultas,int veces)
void pruebaExtremosCategoria(int consultas,int veces)
void enviarGraphvizArbolAvl(std::string arbol)
void enviarGraphvizArbolB(std::string arbol)

void enviarGraphvizArbolBMas(std::string arbol)
void contenidoCsvListo(QString archivo)
void refrescarDatos(bool estado)
void evaluarCerrarCsv(bool estado)
void evaluarErroresLog(bool estado)
void logDescargar(const QString &contenido)
void enviarArbolAvl(NodoAvl * arbol)
void enviarArbolB(NodoB * arbol)
void enviarArbolBMas(NodoBMas * arbol)
void logArbolAvl(QString mensaje, QString color)
void logArbolB(QString mensaje, QString color)
void logArbolBMas(QString mensaje, QString color)
void logLista(QString mensaje, QString color)
void logCargaCsv(QString mensaje, QString color)
void tiempoProceso(int estructura, double milisegundos)
void logInsertArbolAvl(QString mensaje, QString color)
void logInsertArbolB(QString mensaje, QString color)
void logInsertArbolBMas(QString mensaje, QString color)
void logInsertListaOrdenada(QString mensaje, QString color)
void logInsertListaNoOrdenada(QString mensaje, QString color)
void tiempoProcesoInsert (int estructura, double milisegundos)
void logBusquedaNombreArbolAvl(QString mensaje, QString color)
void logBusquedaNombreListaOrdenada(QString mensaje, QString color)
void logBusquedaNombreListaNoOrdenada(QString mensaje, QString color)
void tiempoProcesoBusquedaNombre (int estructura, double milisegundos)
void logBusquedaCategoriaArbolBMas(QString mensaje, QString color)
void logBusquedaCategoriaListaOrdenada(QString mensaje, QString color)

void logBusquedaCategoriaListaNoOrdenada(QString mensaje, QString color)
void tiempoProcesoBusquedaCategoria (int estructura, double milisegundos)
void logBusquedaFechasArbolB(QString mensaje, QString color)
void logBusquedaFechasListaOrdenada(QString mensaje, QString color)
void logBusquedaFechasListaNoOrdenada(QString mensaje, QString color)
void tiempoProcesoBusquedaFechas (int estructura, double milisegundos)
void logEliminarArbolAvl(QString mensaje, QString color)
void logEliminarArbolB(QString mensaje, QString color)
void logEliminarArbolBMas(QString mensaje, QString color)
void logEliminarListaOrdenada(QString mensaje, QString color)
void logEliminarListaNoOrdenada(QString mensaje, QString color)
void tiempoEliminarProceso(int estructura, double milisegundos)
void logListarArbolAvl(QString mensaje, QString color)
void logListarListaOrdenada(QString mensaje, QString color)
void logListarListaNoOrdenada(QString mensaje, QString color)
void tiempoProcesoListarNombre (int estructura, double milisegundos)
void mostrarTiempoPruebasNombre(double milisegundos)
void mostrarTiempoPruebasCategoria(double milisegundos)
void mostrarTiempoPruebasFechas(double milisegundos)

TAD'S DE LA UI

Debido a que se trabajó con una interfaz gráfica esta utiliza el mismo principio de dividir por TAD 'S cada screen por lo tanto se adjunta todo lo utilizado de lógica de UI.

MainWindow

Interfaz Pública:
MainWindow(QWidget *parent = nullptr)
~MainWindow()

Implementación Privada:
Ui::MainWindow *ui
PantallaPrincipal *inicio = nullptr
PantallaAgregar * pantallaAgregar = nullptr
PantallaBuscarPorNombre * pantallaBusquedaNombre = nullptr
PantallaBuscarPorCategoria * pantallaBusquedaCategoria = nullptr
PantallaBuscarRango * pantallaBusquedaRango = nullptr
PantallaEliminarProducto * pantallaEliminar = nullptr
PantallaVerArboles * pantallaVerArboles = nullptr
PantallaListarNombre * pantallaListarNombre = nullptr
Controlador * controladorCrud
void mostrarInicio()
void mostrarAgregar()
void mostrarBuscarNombre()
void mostrarBuscarCategoria()
void mostrarBuscarRango()
void mostrarEliminar()
void mostrarListarNombre()
void mostrarCompararBusqueda()

void mostrarVerArboles()

Implementación de Signals:

void limpiarAgregar()

void limpiarBuscarNombre()

void limpiarBuscarCategoria()

void limpiarBuscarRango()

void limpiarEliminar()

void limpiarListar()

PantallaPrincipal

Interfaz Pública:

explicit PantallaPrincipal(QWidget *parent = nullptr)

~PantallaPrincipal()

Implementación Privada:

bool hayCsv

void descargarLogErrores(const QString &contenido)
--

void limpiarLogs()

Implementación de Signals:

void csvCargado(const std::vector<QString>& data)

void solicitarLogErrores()

void solicitarRefrescarDatos()

void solicitarGuardarCsv()

void solicitarLimpieza()

PantallaPrincipal

Interfaz Pública:
explicit PantallaPrincipal(QWidget *parent = nullptr)
~PantallaPrincipal()

Implementación Privada:
Ui::PantallaPrincipal *ui
bool hayCsv
void descargarLogErrores(const QString &contenido)
void limpiarLogs()

Implementación de Signals:
void csvCargado(const std::vector<QString>& data)
void solicitarLogErrores()
void solicitarRefrescarDatos()
void solicitarGuardarCsv()
void solicitarLimpieza()

PantallaAgregar

Interfaz Pública:
explicit PantallaAgregar(QWidget *parent = nullptr)
~PantallaAgregar()

Implementación Privada:
Ui::PantallaAgregar *ui
void limpiarDatos()
void limpiarLogs()
bool validarCampos()

Implementación de Signals:
void insertarProducto(const std::string &_nombre, const std::string &_codigoBarra, const std::string &_categoria, const std::string &_fechaExpiracion, const std::string &_marca, const std::string &_precio, const std::string &_stock)
void verArboles()
void appendAvlLog(QString mensaje, QString color)
void appendBLog(QString mensaje, QString color)
void appendBMasLog(QString mensaje, QString color)
void appendListOrdenadaLog(QString mensaje, QString color)
void appendListNoOrdenadaLog(QString mensaje, QString color)
void mostrarTiempo(int estructura, double milisegundos)
void limpiarPantalla()

PantallaBuscarPorNombre

Interfaz Pública:
explicit PantallaBuscarPorNombre(QWidget *parent = nullptr)
~PantallaBuscarPorNombre()

Implementación Privada:
Ui::PantallaBuscarPorNombre *ui
void limpiarDatos()
void limpiarLogs()
bool validarPruebas(int &consultas, int &veces)

Implementación de Signals:
void buscarPorNombre(std::string nombre)
void verArboles()
void pruebaAleatoria(int consultas,int veces)
void pruebaExtremos(int consultas,int veces)
void appendAvlLog(QString mensaje, QString color)
void appendListOrdenadaLog(QString mensaje, QString color)
void appendListNoOrdenadaLog(QString mensaje, QString color)
void mostrarTiempo(int estructura, double milisegundos)
void mostrarTiempoPruebas(double milisegundos)
void limpiarPantalla()

PantallaBuscarPorCategoria

Interfaz Pública:
explicit PantallaBuscarPorCategoria(QWidget *parent = nullptr)
~PantallaBuscarPorCategoria()

Implementación Privada:
Ui::PantallaBuscarPorCategoria *ui
void limpiarDatos()
void limpiarLogs()
bool validarPruebas(int &consultas, int &veces)

Implementación de Signals:
void buscarPorCategoria(std::string nombre)
void verArboles()
void pruebaAleatoria(int consultas,int veces)
void pruebaExtremos(int consultas,int veces)
void appendBMasLog(QString mensaje, QString color)
void appendListOrdenadaLog(QString mensaje, QString color)
void appendListNoOrdenadaLog(QString mensaje, QString color)
void mostrarTiempo(int estructura, double milisegundos)
void mostrarTiempoPruebas(double milisegundos)
void limpiarPantalla()

PantallaBuscarRango

Interfaz Pública:
explicit PantallaBuscarRango(QWidget *parent = nullptr)
~PantallaBuscarRango()

Implementación Privada:
Ui::PantallaBuscarRango *ui
void limpiarDatos()
void limpiarLogs()
bool validarPruebas(int &consultas, int &veces)

Implementación de Signals:
void buscarPorFechas(std::string limiteInferior, std::string limiteSuperior)
void verArboles()
void pruebaAleatoria(int consultas,int veces, const std::string &limiteInferior, const std::string &limiteSuperior)
void pruebaExtremos(int consultas,int veces)
void appendBLog(QString mensaje, QString color)
void appendListOrdenadaLog(QString mensaje, QString color)
void appendListNoOrdenadaLog(QString mensaje, QString color)
void mostrarTiempo(int estructura, double milisegundos)
void mostrarTiempoPruebas(double milisegundos)
void limpiarPantalla()

PantallaEliminarProducto

Interfaz Pública:
explicit PantallaEliminarProducto(QWidget *parent = nullptr)
~PantallaEliminarProducto()

Implementación Privada:
Ui::PantallaEliminarProducto *ui
void limpiarDatos()
void limpiarLogs()

Implementación de Signals:
void eliminarProducto(std::string codigo)
void verArboles()
void appendAvlLog(QString mensaje, QString color)
void appendBLog(QString mensaje, QString color)
void appendBMasLog(QString mensaje, QString color)
void appendListOrdenadaLog(QString mensaje, QString color)
void appendListNoOrdenadaLog(QString mensaje, QString color)
void mostrarTiempo(int estructura, double milisegundos)
void limpiarPantalla()

PantallaListarNombre

Interfaz Pública:
explicit PantallaListarNombre(QWidget *parent = nullptr)
~PantallaListarNombre()

Implementación Privada:
Ui::PantallaListarNombre *ui
void limpiarLogs()

Implementación de Signals:
void verArboles()
void appendAvlLog(QString mensaje, QString color)
void appendListOrdenadaLog(QString mensaje, QString color)
void appendListNoOrdenadaLog(QString mensaje, QString color)
void mostrarTiempo(int estructura, double milisegundos)
void limpiarPantalla()

PantallaVerArboles

Interfaz Pública:
explicit PantallaVerArboles(QWidget *parent = nullptr)
~PantallaVerArboles()
void reestablecerVistas()

Implementación Privada:
Ui::PantallaVerArboles *ui
void limpiarLogs()

Implementación de Signals:
void solicitarArbolAvl()
void solicitarArbolB()
void solicitarArbolBMas()
void solicitarGraphArbolAvl()
void solicitarGraphArbolB()
void solicitarGraphArbolBMas()
void recibirArbolAvl(NodoAvl * arbol)
void recibirArbolB(NodoB * arbol)
void recibirArbolBMas(NodoBMas * arbol)
void recibirGrapvizAvl(std::string dot)
void recibirGrapvizB(std::string dot)
void recibirGrapvizBMas(std::string dot)

PantallaArbolAvl

Interfaz Pública:
explicit PantallaArbolAvl(QWidget *parent = nullptr)
~PantallaArbolAvl()
void setArbol(NodoAvl *_arbol)
void generarGraphviz(std::string graph)

Implementación Privada:
Ui::PantallaArbolAvl *ui
QGraphicsScene * scene = nullptr
NodoAvl * arbol = nullptr
void actualizarVista()
QRectF dibujarNodo(int x, int y, QString texto, int altura)
void dibujarLinea(int x1, int y1, int x2, int y2)
void dibujarArbol(NodoAvl* nodo, int x, int y, int offset)

Implementación de Signals:
void solicitarArbolAvl()
void solicitarGraphvizAvl()

PantallaArbolB

Interfaz Pública:
explicit PantallaArbolB(QWidget *parent = nullptr)
~PantallaArbolB()
void setArbol(NodoB * _arbol)
void generarGraphviz(std::string graph)

Implementación Privada:
Ui::PantallaArbolB *ui
QGraphicsScene * scene = nullptr
GraphicsViewZoom *zoomView = nullptr
NodoB * raiz = nullptr
int dibujarNodoB(int x, int y, NodoB* nodo)
void dibujarLineaB(int x1, int y1, int x2, int y2)
void dibujarArbolB(NodoB* nodo, int x, int y)
void actualizarVista()
int getAnchoTotal(NodoB* nodo)
int getAnchoNodoReal(NodoB* nodo)

Implementación de Signals:
void solicitarArbolB()
void solicitarGraphvizB()

PantallaArbolBMas

Interfaz Pública:
explicit PantallaArbolBMas (QWidget *parent = nullptr)
~ PantallaArbolBMas ()
void setArbol(NodoBMas * _arbol)
void generarGraphviz(std::string graph)

Implementación Privada:
Ui::PantallaArbolBMas *ui
QGraphicsScene * scene = nullptr
GraphicsViewZoom *zoomView = nullptr
NodoBMas * raiz = nullptr
std::unordered_map<NodoBMas*, int> posicionesX
std::unordered_map<NodoBMas*, int> posicionesY
void actualizarVista()
int dibujarNodoBMas(int x, int y, NodoBMas* nodo)
void dibujarLineaBMas(int x1, int y1, int x2, int y2)
void dibujarArbolBMas(NodoBMas* nodo, int x, int y)
void dibujarConexionesHojas(NodoBMas* nodoRaiz)
int getAnchoNodoReal(NodoBMas* nodo)
int getAnchoTotal(NodoBMas* nodo)

Implementación de Signals:
void solicitarArbolBMas()
void solicitarGraphvizBMas()